

Statistics 6910: Writing Your Own Functions (1)

So far we have relied on the built-in functionality of R to carry out our analyses. In the next several lectures, we will cover:

- How to write your own functions
- How to use flow control mechanisms like if and for
- Debugging your code when something goes wrong
- The meaning of “environments” and “variable scope”
- Timing and speeding up your code

Writing your own functions in R

Think about the code we have been writing so far in R. It has been

- made up of a sequence of commands, one after another
- specific to the particular dataset we’re working with.

Functions allow us to

- organize our code into tasks
- reuse the same code on different datasets by making the data an *argument* to the function.

Our task is to convert inches into centimeters for family height:

```
family$height * 2.54
```

We can make a conversion function InToCm function that looks like:

```
Convert = function(x) x*2.54
```

This function *encapsulates* the multiplication of each element by 2.54 for *any* x vector, not just the family\$height.

Anatomy of a function

The syntax for writing a function is

```
function (arguments) body
```

Typically we assign the function to a particular name. This should describe what the function does.

```
myFunction = function (arguments) body
```

A function without a name is called an “orphan” function. These can be very powerfully used with the apply mechanism as seen in the previous section.

```
function (arguments) body
```

The keyword `function` just tells R that you want to create a function.

Recall that the *arguments* to a function are its inputs, which may have default values.

```
> args(median)
function (x, na.rm = FALSE)
```

Here, if we do not explicitly specify `na.rm` when we call `median`, it will be assigned the default value of `FALSE`.

A few notes on specifying the arguments:

When you are writing your own function, it is good practice to put the most important arguments first. Often, these will not have default values.

This allows the user of your function to easily specify the arguments by position, e.g., `plot(xvec, yvec)` rather than `plot(x = xvec, y = yvec)`.

Next we have the *body* of the function, which typically consists of expressions surrounded by curly brackets. Think of these as performing some operations on the input values given by the arguments.

```
{
  expression 1
  expression 2
  return(value)
}
```

The `return` expression hands control back to the caller of the function and returns a given `value`. If the function returns more than one thing, this is done using a named list, for example

```
return(list(total = sum(x), avg = mean(x)))
```

In the absence of a return expression, a function will return the *last* evaluated expression. This is particularly common if the function is short.

That is the case for our simple function:

```
Convert = function(x) x * 2.54
```

Here we don't need brackets {}, since there is only one expression.

A return expression anywhere in the function will cause the function to return control to the user *immediately*, without evaluating the rest of the function. This is often used in conjunction with `if` statements, which we will discuss later.

Considerations when writing a function:

- What will the function do?
- What should we call it? (Relate the name to what it does)
- What will be the arguments?
- Which arguments have default values and what are they?
- What (if anything) should the function return?

THE END

More to come in
Stat 6910, Section 004
(Advanced R)